GCP Resource Tagging:
Eliminating Custom Code via a Cloud-Native Approach

Prepared for: RB Global — Cloud Engineering | June 2026

1. Challenge: Custom Cloud Run Application Introduces Unscoped Technical Debt

The current implementation approach proposes a custom-built Cloud Run Tag Compliance Evaluator requiring development, containerisation, and indefinite maintenance of a bespoke application — work that falls outside the agreed scope of this engagement.

Custom Development Required	Ongoing Maintenance Burden
Python/Go application (~1000+ lines)	Library & SDK version upgrades
Dockerfile + container build pipeline	Container image CVE patching
Per-resource-type API routing logic (8 types)	Regression testing on GCP API changes
Authentication & retry handling	CI/CD pipeline maintenance
Unit + integration test suite	Incident response & monitoring

Key Concerns

Concern	Detail
Out of Scope	The agreed objective is achieving label compliance — not delivering a custom software product. Writing and maintaining a Cloud Run application was not part of the scoped deliverables.
Technical Debt	Every custom line of code requires indefinite maintenance: security patches, SDK upgrades, dependency conflicts, and runtime failures — regardless of whether new features are being built.
Operational Overhead	Container registries, build pipelines, health checks, scaling configuration, and on-call incident response are introduced from day one — none of which apply to a native GCP approach.

2. Recommended Alternative: GCP-Native Services + gcloud CLI

The same compliance outcomes are achievable without writing any application code, using exclusively managed GCP services and standard tooling across three layers.

Layer 1 — Prevention (Greenfield): Terraform + Organisation Policy

Mechanism	What It Does	GCP Reference
Terraform labels block	Applies all mandatory labels at resource creation time for every IaC-managed resource — zero runtime logic required.	Native Terraform attribute
Org Policy Custom Constraints (CEL)	Blocks resource creation at the GCP API layer if mandatory labels are absent — enforced equally for Console, CLI, and API callers.	cloud.google.com/.../custom-constraints
IAM Deny Policies	Prevents label removal by non-authorised principals after labels are applied — no application code required.	cloud.google.com/iam/docs/deny-overview

Layer 2 — Detection (Brownfield): Cloud Asset Inventory

Mechanism	What It Does	GCP Reference
CAI Export to BigQuery	Scheduled export of all resource metadata (including labels) org-wide to BigQuery — one gcloud command, no code.	cloud.google.com/.../exporting-to-bigquery
BigQuery SQL	Standard SQL identifies resources with missing or non-compliant labels. GCP provides sample queries out of the box.	cloud.google.com/.../sample-queries

Layer 3 — Remediation (Brownfield): Two Scenarios

Brownfield resources fall into two categories, each remediated via a different native mechanism — no custom application required in either case.

Scenario A: Terraform-Managed Resources

For resources already under Terraform state management, labels are added through the standard IaC workflow.

Step	Action	Detail
1	**Add labels block**	Add the five mandatory label keys (values from Application Registry) to the resource .tf configuration.
2	**terraform plan**	Confirms labels will be added with no destructive changes. For Compute, Storage, BigQuery, Cloud SQL, and GKE, label updates are in-place and do not trigger resource recreation.
3	**terraform apply**	Applies labels across all affected resources. A shared rbi-labels module can centralise value resolution from the Application Registry, reducing per-resource effort to a single module reference.

Note: For most GCP resource types (Compute, Storage, BigQuery, Cloud SQL, GKE), updating labels does not trigger resource recreation — it is an in-place metadata update. Source: GCP Terraform Provider documentation.

Scenario B: Non-Terraform Resources (Console-created, CLI-created, or Unmanaged)

For resources not tracked in any Terraform state, labels are applied using native gcloud commands. The --update-labels flag is additive: it only adds or updates specified keys without removing existing labels (native no-overwrite behaviour — Source: gcloud CLI Reference).

Resource Type	Native gcloud Command
Compute Instance	*gcloud compute instances update INSTANCE --update-labels=owner=X,application=Y,...*
Compute Disk	*gcloud compute disks update DISK --update-labels=owner=X,application=Y,...*
Cloud Storage Bucket	*gcloud storage buckets update gs://BUCKET --update-labels=owner=X,application=Y,...*
BigQuery Dataset	*bq update --set_label owner:X --set_label application:Y project:dataset*
Cloud SQL	*gcloud sql instances patch INSTANCE --update-labels=owner=X,application=Y,...*
GKE Cluster	*gcloud container clusters update CLUSTER --update-labels=owner=X,application=Y,...*
Pub/Sub Topic	*gcloud pubsub topics update TOPIC --update-labels=owner=X,application=Y,...*

Execution Options for Non-Terraform Remediation

Option	How	Best For
Manual	Operator runs gcloud commands per resource from the BigQuery non-compliance report.	Small batches; initial one-time remediation.
Shell Script	Loop over a BigQuery CSV export; execute gcloud per resource row.	Bulk remediation — an operational script, not a maintained application.

3. Approach Comparison

Criteria	Cloud Run (Custom Code)	Native + gcloud (Proposed)
Application code	500-800 lines Python/Go	None
Container management	Required	Not applicable
CI/CD build pipeline	Required	Not required
Security patching	Team responsibility	GCP-managed
Ongoing developer capacity	Required indefinitely	Not required
Terraform resource remediation	Custom API calls in code	terraform apply (native)
Non-Terraform remediation	Custom API calls in code	gcloud --update-labels (native CLI)
Brownfield detection	Custom CAI event handler	CAI Export + BigQuery SQL (native)
Greenfield prevention	Must be coded separately	Org Policy + Terraform (native)
Label protection	Must be coded separately	IAM Deny Policies (native)

4. Recommendation

We recommend proceeding with the GCP-native approach. Every compliance layer is addressed using managed services and standard tooling — zero custom application code required.

Compliance Layer	Native Approach	Custom Code?
Prevention	Terraform labels module + Org Policy Custom Constraints	**No**
Detection	Cloud Asset Inventory Export to BigQuery + SQL queries	**No**
Remediation (Terraform)	Add labels block to .tf config, then terraform apply	**No**
Remediation (Non-Terraform)	gcloud --update-labels via manual run or shell script	**No**
Label Protection	IAM Deny Policies at organisation level	**No**
Compliance Reporting	Looker Studio connected to BigQuery compliance dataset	**No**

This approach delivers the same compliance outcome with **zero custom application code**, no container infrastructure, and no long-term technical debt.

> **Note — WRONG_VALUE resources:** Resources where a label key exists but holds an incorrect value cannot be auto-corrected by any native service. These are surfaced via the BigQuery compliance report and require manual correction by the resource owner. This is an operational process, not a software development task.

References: GCP Cloud Asset Inventory Docs | Organisation Policy Custom Constraints | IAM Deny Policies Overview | gcloud CLI Reference | GCP Terraform Provider Docs